Assignment - 07

Title : Graph Minimum Spanning Tree.

Objective : To Study Some advanced data Structure Such as Graph's.

Problem Statement :
Represent a graph of your college campus using adjacency matrix. Node's should represent the various departments / institutes and links should represent the distance between them. Find minimum Spanning tree.

a) Using Kruskal's algorithm.
b) Using prim's algorithm.

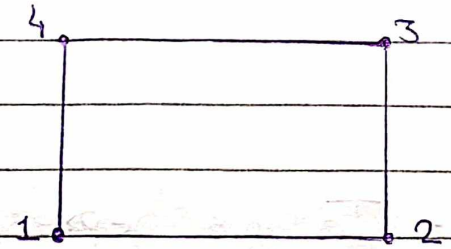
Outcome :- Solve problem using algorithmic design technique's and data Structure's.

Theory :

Graph's :- A graph is a Collection of two set V and E where V is Finite non-empty set of vertice's and E is Finite non-empty set of edge's.

- Vertice's are nothing but the node's in the graph.
- Two adjacent vertice's are joined by edge's.
- Any graph is denoted as $G = \{V, E\}$

For e.g :-



$V(G) = \{1, 2, 3, 4\}$

$E(G) = \{(1,2)(1,4)(2,3)(3,4)\}$

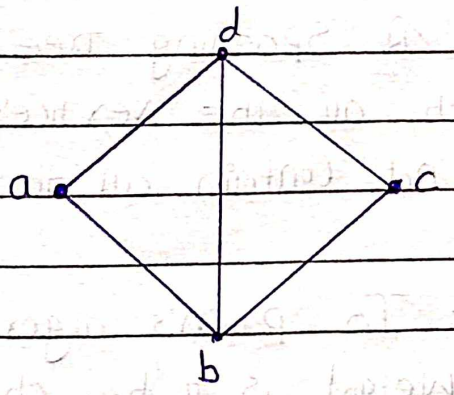
Normally a graph can be represented by two representation's and those are

1). Adjacency Matrix
2). Adjacency list

1). Adjancecy Matrix :- In this representation, Matrix of 1or 2-D array is used to represent the graph.

Consider a Graph G of n vertice's and the matrix M. if there is an edge present between V_i and V_j then $M_{ij} = 1$ else $M[i][j] = 0$.

e.g :-

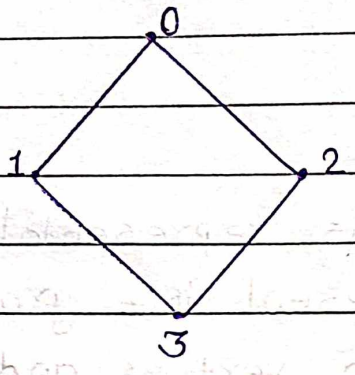


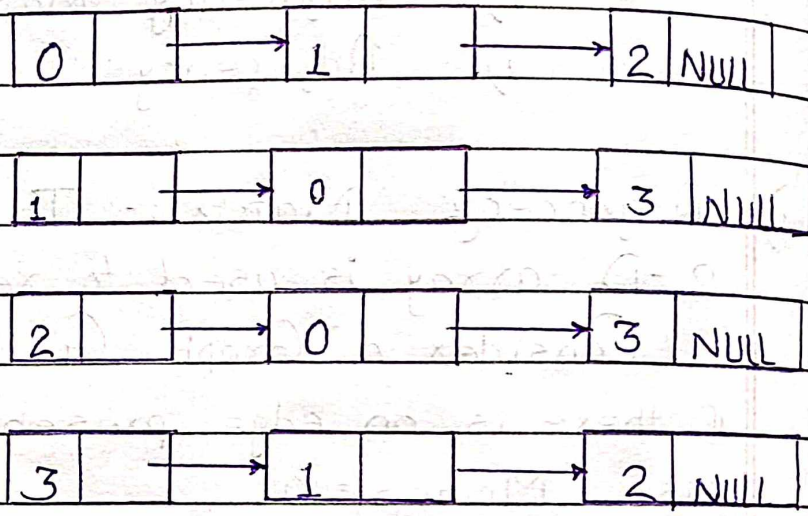
	a	b	c	d
a	0	1	0	1
b	1	0	1	1
c	0	1	0	1
d	1	1	1	0

Adjacency matrix for Weighted graph :- In the Weighted graph, Weight's are assign along every edge's, an adjacency matrix representation any edge which is present between vertice's V_i and V_j is denoted by it's Weight. Hence $M[i][j]$ = Weight of edge. If there is no edge bet^n V_i and V_j then, $M[i][j] = 0$.

2). Adjacency list ;- In this representation, a linked list is used to represent a graph.

e.g ;-

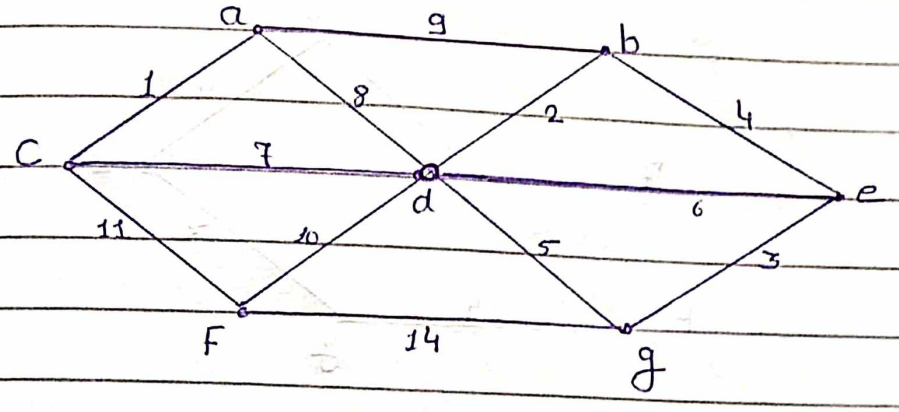




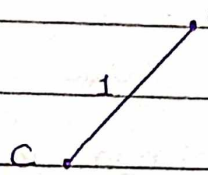
Spanning Tree ;- A Spanning tree S is a subset of a tree T in which all the vertice's of tree T are present but it may not contain all the edge's.

i) Prim's Algorithm ;- In prim's algorithm, the pair with the minimum weight is to be chosen. Then adjacent to these vertice's which ever is the edge having minimum weight is selected. This process will be continued till all the vertice's are not be covered. The necessary condition in this case is that the circuit should not be formed.

e.g :-

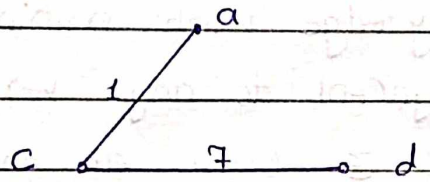


Step 1 :



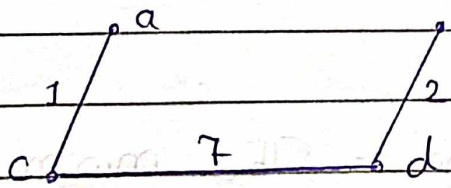
path length = 1

Step 2 :



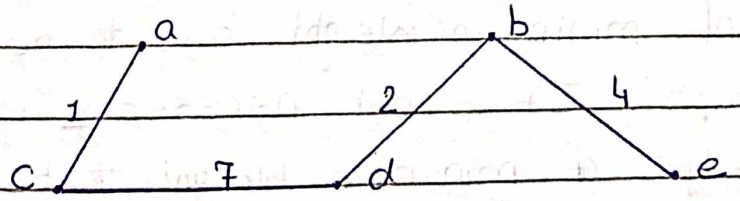
path length = 8

Step 3 :



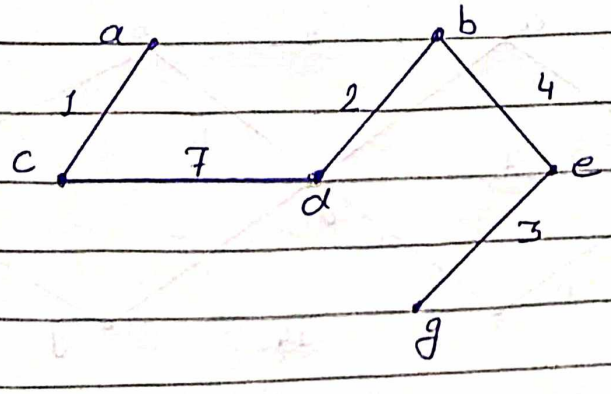
path length = 10

Step 4 :



Path length = 14

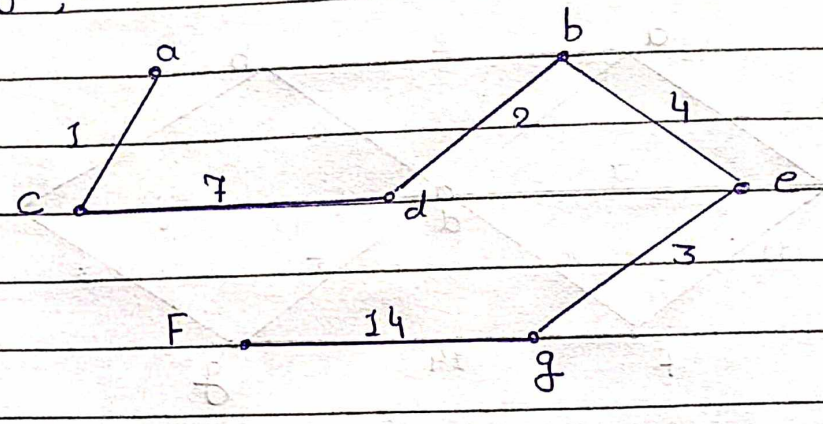
Step 5 :



Path length = 17

Step 6 :-

Path length = 31.

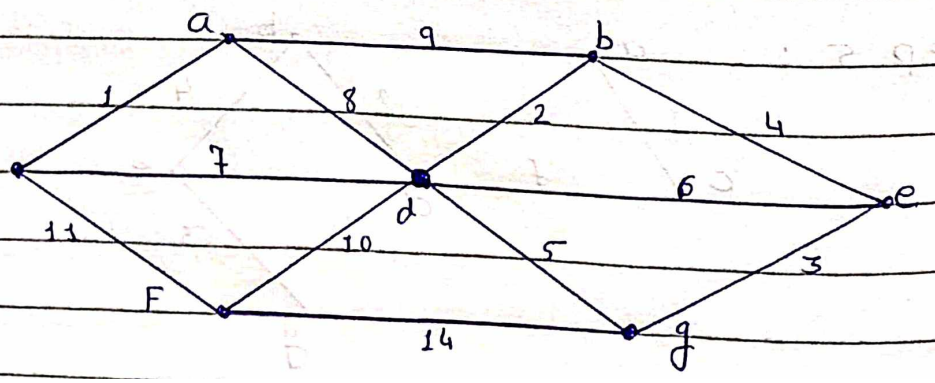


Algorithm :-

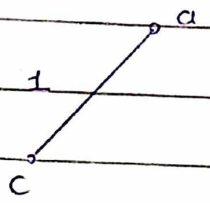
1. Select an edge with minimum Weight.
2. Add the vertice's in the set V.
3. Then select any edge with minimum Weight such that the edge is adjacent to any vertex from set V.
4. Repeat step 2, 3 until all the vertice's are selected. But circuit should not be formed while seleting this vertice's.

2). Kruskal's Algorithm :- The minimum Weight is obtained and also circuit should not be formed. Each time the edge of minimum Weight has to be selected, from the graph. It is not necessary in this algorithm to have edge's of minimum Weight to be adjacent.

e.g :-



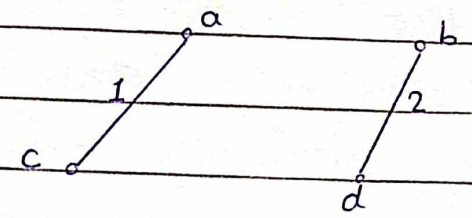
Step 1 :-



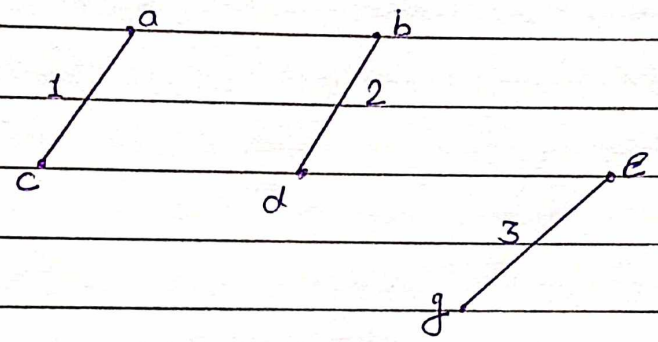
This is the graph with mi
Spanning tree and it's tota
Weight is,

1 + 7 + 2 + 4 + 3 + 10 =

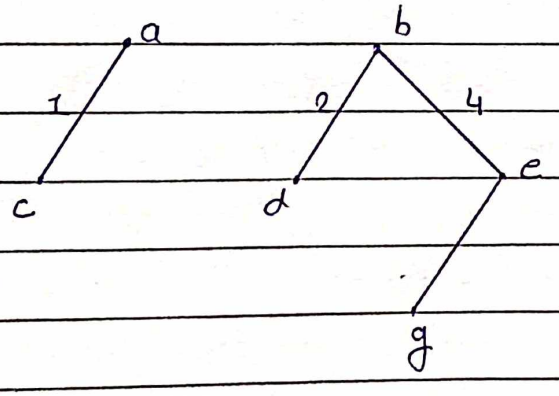
Step 2 :-



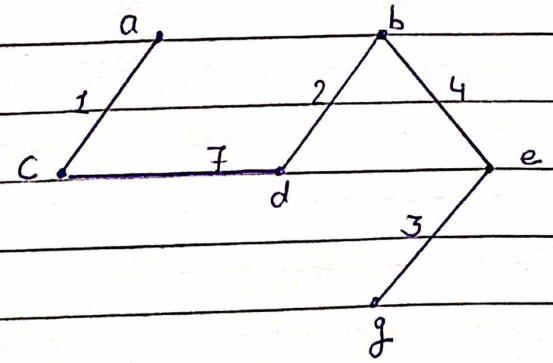
Step 3 :-



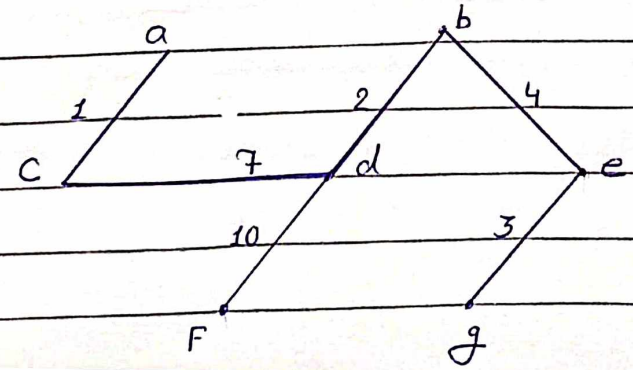
Step 4 :-



Step 5 :-



Step 6 :-



Algorithm :

1. Sort all edge's in increasing order of their edge Weigth.
2. Pick the Smallest edge.
3. Check if the new edge create's an circuit or loop in a Spanning tree.
4. If it doesn't form the Cycle, then include that edge in MST, other wise discard it.
5. Repeat from Step no. 2 until it include's |v|-1 edge's in MST.

Test Case	Test Case Description	Test Data	Expected Result	Actual Result	Pass/ Fail
1.	Check result When user Select an option of prim's algorithm for MST	Find out MST by using prim's algorithm	The Min. Spanning tree get obtained by using prim's algorithm	Min. Spanning tree get obtained by using prim's algorithm	Pass
2.	Check result When user Select an option of kruskal's algorithm for MST.	Find out MST using kruskal's algorithm	Min. Spanning tree Should obtained using kruskal algorithm	Min. Spanning tree get's obtained using kruskal algorithm.	Pass

Assignment 7

```
#include<iostream>
#include<iomanip>
#define SIZE 100
using namespace std;

class Graph
{
   int vertices,edges,cost;
   int graph[SIZE][SIZE];
   int mst[SIZE][SIZE];
   int selected[SIZE][SIZE];
         int parent[SIZE];
   int distance[SIZE];
   int visited[SIZE];

   public:
      Graph(){}
      Graph(int,int);
      void create();
      void display();
      void prims();
      int findparent(int v);
      void kruskal();
      void displaymst();
};

Graph::Graph(int v,int e)
{
```

```
    vertices = v;
    edges = e;
    cost=0;

    for(int i=0;i<vertices;i++)
    {
        distance[i]=INT32_MAX;
        visited[i]=0;
        parent[i]=i;
        for(int j=0;j<vertices;j++)
        {
            graph[i][j]=0;
            selected[i][j]=0;
            mst[i][j]=0;
        }
    } distance[0]=0;
}

void Graph::create()
{
    int source,destination,weight;
    for(int i=0;i<edges;i++)
    {
        cout<<"\nEnter the source vertex:- ";
        cin>>source;
        cout<<"Enter the destination vertex:- ";
        cin>>destination;

        if(source != destination)
        {
            if(graph[source-1][destination-1]==0 && graph[source-1][destination-1]==0)
```

```
        {
            cout<<"Enter the weight of the graph:- ";
            cin>>weight;
            graph[source-1][destination-1] = weight;
            graph[destination-1][source-1] = weight;
            cout<<"Inserted edge between "<<source<<" and "<<destination<<endl;
        }
        else
        {
            cout<<"\nEdge already exists. Please select a new edge"<<endl;
            i--;
            continue;
        }
    }
    else
    {
        cout<<"\nSource and destination cannot be the same\n";
        i--;
        continue;
    }

  }
  cout<<"\n\nGraph created successfully"<<endl;
}

void Graph::display()
{
  for(int i=0;i<vertices;i++)
  {
    for(int j=0;j<vertices;j++)
        cout<<graph[i][j]<<"  ";
```

```
        cout<<endl;
    }
}

int Graph::findparent(int v)
{
        if(parent[v] == v)
                return v;
        return findparent(parent[v]);
}
void Graph::prims()
{
   int min;

   for (int k = 0; k < vertices - 1; k++)
   {
      min = -1;
      for (int i = 0; i < vertices; i++)
      {
         if (!visited[i] && (min == -1 || distance[i] < distance[min]))
            min = i;
      }
      visited[min] = 1;

      for (int i = 0; i < vertices; i++)
      {
         if (graph[min][i] != 0 && !visited[i] && graph[min][i] < distance[i])
         {
            distance[i] = graph[min][i];
            parent[i] = min;
         }
```

```
    }
  }

  for (int i = 0; i < vertices; i++)
  {
    int j = parent[i];
    if (j != -1)
    {
      mst[j][i] = distance[i];
      mst[i][j] = distance[i];
      cost += distance[i];
    }
  }
}

void Graph::kruskal()
{
  int min_weight,min_source,min_destination;
  int k=1;
  cost=0;

  while (k!=vertices)
  {
    min_weight=100;
    for(int i=0;i<vertices;i++)
    {
      for(int j=0;j<vertices;j++)
      {
        if(graph[i][j] && !selected[i][j] && graph[i][j] <= min_weight)
        {
          min_weight = graph[i][j];
```

```
            min_source = i;
            min_destination = j;
          }
        }
      }
      if(findparent(min_source) != findparent(min_destination))
                {
                        mst[min_source][min_destination] = min_weight;
                        mst[min_destination][min_source] = min_weight;
                        parent[min_destination] = min_source;
                        cost += mst[min_source][min_destination];
                        selected[min_source][min_destination] = 1;
                        selected[min_destination][min_source] = 1;
                        k++;
                }
        }
}

void Graph::displaymst()
{
        cout<<"\nThe minimum spanning tree is:- "<<endl;

        for(int i=0; i<vertices; i++)
        {
                for(int j=0; j<vertices; j++)
                        cout<<mst[i][j]<<" ";
                cout<<endl;
        }
        cout<<"\nThe cost of the MST is : "<<cost<<endl;
}
```

```
int main()
{
        Graph g;
  int choice, e, v;

        while(1)
  {

        cout<<"\nImplementation of MST"<<endl;
    cout<<"1. Create graph"<<endl;
    cout<<"2. Display graph"<<endl;
    cout<<"3. Find MST using Kruskal's algorithm"<<endl;
    cout<<"4. Find MST using prim's algorithm"<<endl;
    cout<<"5. Exit the program"<<endl;
    cout<<"\nEnter your choice:- ";
    cin>>choice;

                switch(choice)
                {
      case 1:
        cout<<"\nEnter the number of vertices:-";
        cin>>v;
        cout<<"\nEnter the number of edges:-";
        cin>>e;
        g = Graph(v,e);
        g.create();
        break;

      case 2:
        g.display();
```

```
            break;

        case 3:
            g.kruskal();
            g.displaymst();
            break;

        case 4:
            g.prims();
            g.displaymst();
            break;

        case 5:
            return 0;

        default:
            cout<<"\nError in choice, try again"<<endl;
                    }
          }
   return 0;
}
```